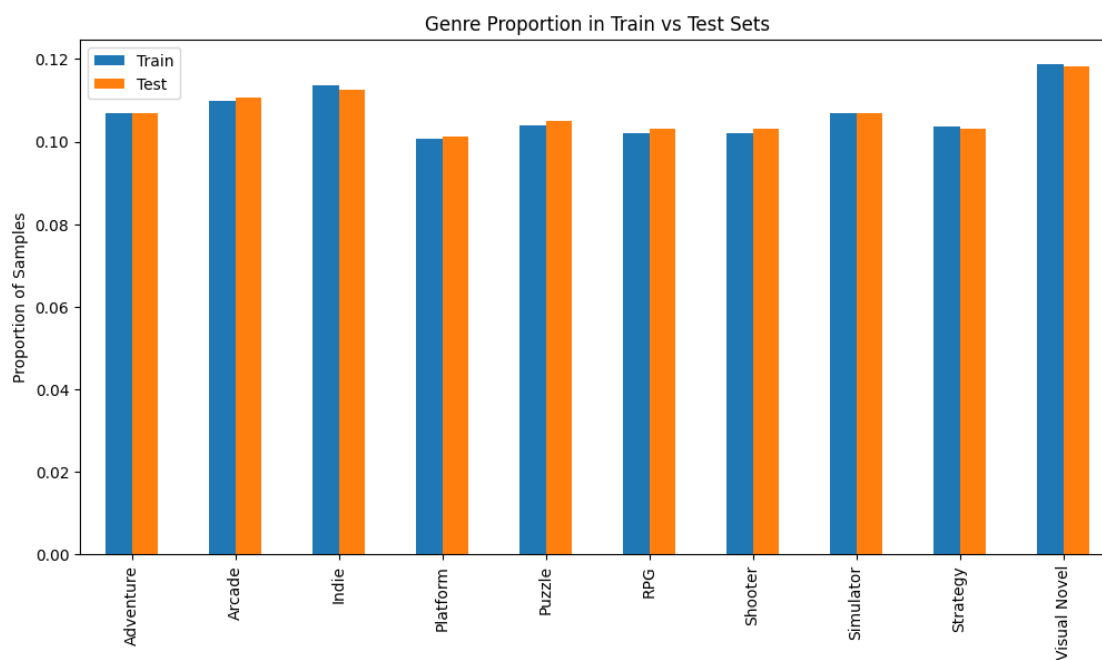


green26_0-4

June 18, 2026

```
[9]: #EXPORT
import matplotlib.pyplot as plt
df = pd.DataFrame({
    'Train': train_props,
    'Test': test_props
})
df.plot(kind='bar', figsize=(12, 6), title="Genre Proportion in Train vs Test_
↪Sets")
plt.ylabel("Proportion of Samples")
plt.show()
```



```
[13]: #EXPORT
training_info = (f"Final training accuracy: {training.
↪history['accuracy'][-1]}\n"
                f"Final training loss: {training.history['loss'][-1]}\n")
```

```

        f"Final validation accuracy: {training.
↪history['val_accuracy'][-1]}\n"
        f"Final validation loss: {training.history['val_loss'][-1]}\n")
fig, ax = plt.subplots(figsize=(4, 1))
ax.axis('off') # hide axes
ax.text(0.1, 0.98, training_info, fontsize=10, va='top', family='monospace')
plt.show()

```

```

Final training accuracy: 0.30071598291397095
Final training loss: 0.48325029015541077
Final validation accuracy: 0.2571428716182709
Final validation loss: 0.5034171938896179

```

```

[15]: #EXPORT
#strict multi-label accuracy, model is only "correct" if it gets all the genres
↪for a game right
strict_acc = accuracy_score(y_test, y_pred)

#hamming loss, evaluates miss classification by individual genre label
hamming = hamming_loss(y_test, y_pred)

# Macro and micro averaged F1 score
f1_macro = f1_score(y_test, y_pred, average='macro')
f1_micro = f1_score(y_test, y_pred, average='micro')

#generating the report
#high precision means few false positives, high recall means few false negatives
report = classification_report(y_test, y_pred, target_names = genre_names)

other_info = (f"Strict multi-label accuracy (exact match): {strict_acc}\n"
              f"Hamming loss: {hamming}\n"
              f"F1 score (macro): {f1_macro}\n"
              f"F1 score (micro): {f1_micro}\n"
              f"\nClassification Report:\n")

#saving it as a figure with text
fig, ax = plt.subplots(figsize=(8, 4))
ax.axis('off') # hide axes
ax.text(0.1, 0.98, other_info, fontsize=10, family="monospace")
ax.text(0.1, 0.98, report, fontsize=10, va='top', family='monospace')
plt.show()

```

```

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in samples

```

with no predicted labels. Use `zero\_division` parameter to control this behavior.

```
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

```
Strict multi-label accuracy (exact match): 0.1049618320610687
Hamming loss: 0.16564885496183207
F1 score (macro): 0.24150197750730024
F1 score (micro): 0.2517241379310345
```

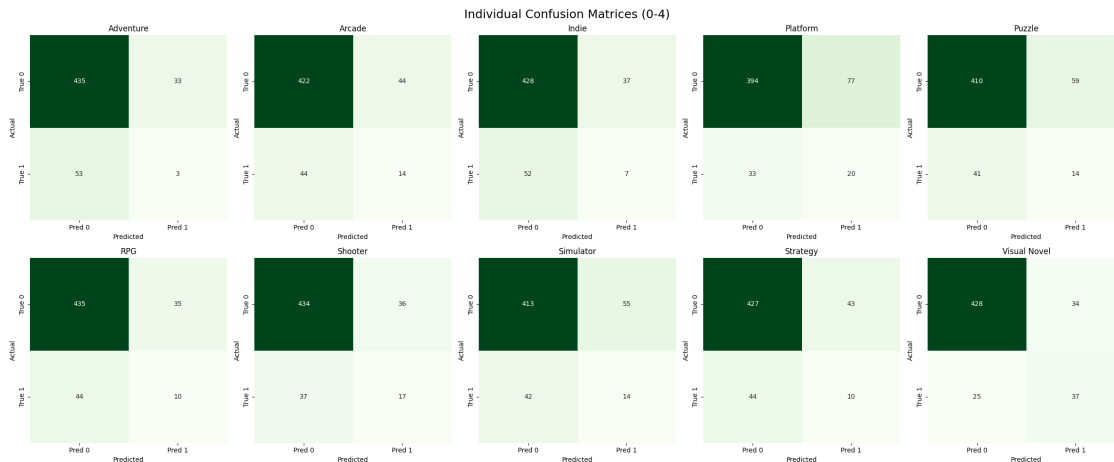
Classification Report:

	precision	recall	f1-score	support
Adventure	0.08	0.05	0.07	56
Arcade	0.24	0.24	0.24	58
Indie	0.16	0.12	0.14	59
Platform	0.21	0.38	0.27	53
Puzzle	0.19	0.25	0.22	55
RPG	0.22	0.19	0.20	54
Shooter	0.32	0.31	0.32	54
Simulator	0.20	0.25	0.22	56
Strategy	0.19	0.19	0.19	54
Visual Novel	0.52	0.60	0.56	62
micro avg	0.24	0.26	0.25	561
macro avg	0.23	0.26	0.24	561
weighted avg	0.24	0.26	0.24	561
samples avg	0.19	0.26	0.21	561

```
[16]: #EXPORT
#Plotting individual confusion matrices for each genre
mlcm = multilabel_confusion_matrix(y_test, y_pred)
class_labels = [f"Label {i}" for i in range(mlcm.shape[0])]

individual, axes = plt.subplots(nrows=2, ncols=5, figsize=(24, 10))
individual.suptitle("Individual Confusion Matrices (0-4)", fontsize=18)
for i, cm in enumerate(mlcm):
    row, col = divmod(i, 5)
    ax = axes[row, col]
    sns.heatmap(cm, annot=True, fmt='d', cmap='Greens', cbar=False,
                xticklabels=["Pred 0", "Pred 1"],
                yticklabels=["True 0", "True 1"],
                ax=ax)
    ax.set_title(f"{genre_names[i]}")
    ax.set_xlabel("Predicted")
    ax.set_ylabel("Actual")
plt.tight_layout()
```

```
plt.show()
```

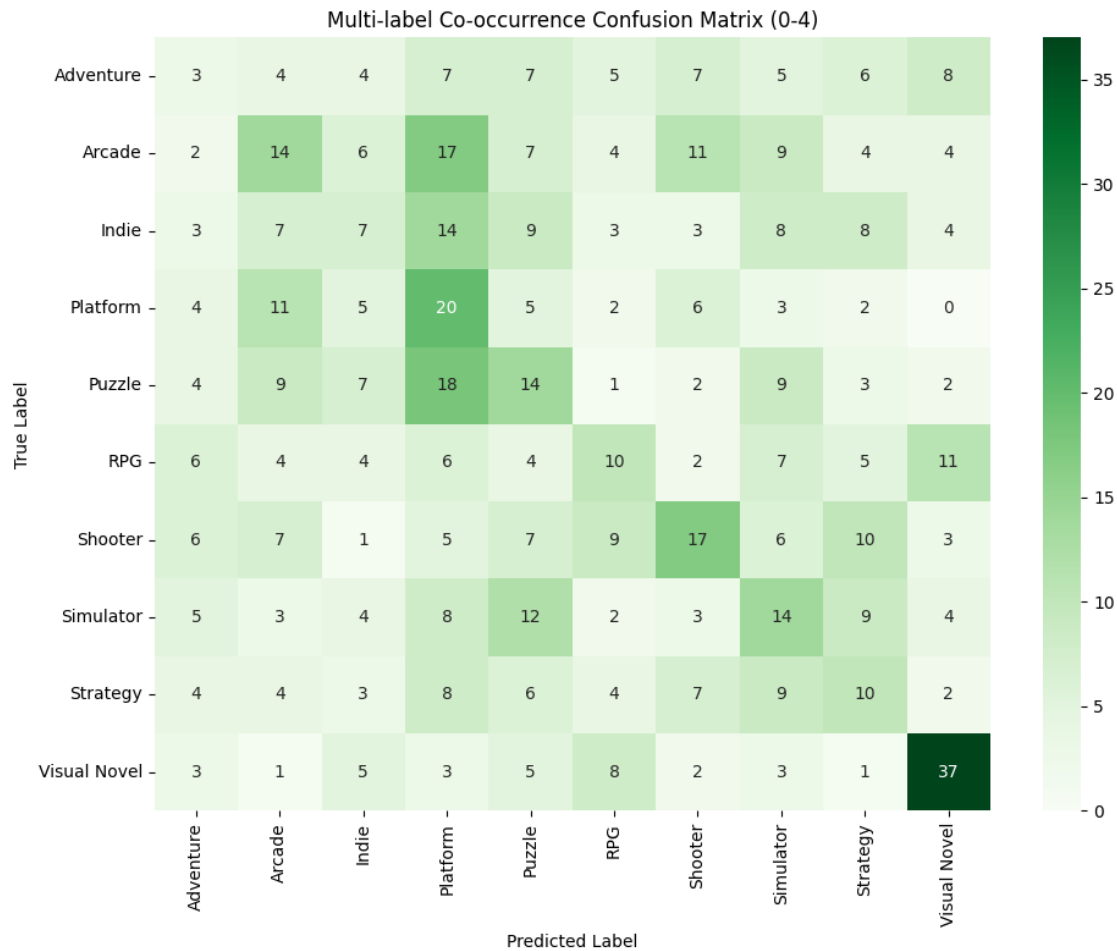


```
[17]: #EXPORT
#Generate a co-occurrence confusion matrix
#Each row is a true label vector, each column is a predicted label vector.
#This gives you a sense of how many labels were mispredicted together.

def multilabel_confusion_counts(y_true, y_pred):
    co_occurrence = np.zeros((y_true.shape[1], y_pred.shape[1]), dtype=int)
    for i in range(len(y_true)):
        true_indices = np.where(y_true[i] == 1)[0]
        pred_indices = np.where(y_pred[i] == 1)[0]
        for t in true_indices:
            for p in pred_indices:
                co_occurrence[t, p] += 1
    return co_occurrence

#Generate and plot the co-occurrence matrix
co_matrix = multilabel_confusion_counts(y_test, y_pred)

multilabel = plt.figure(figsize=(10, 8))
sns.heatmap(co_matrix, annot=True, fmt='d', cmap='Greens',
            xticklabels=genre_names, yticklabels=genre_names)
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Multi-label Co-occurrence Confusion Matrix (0-4)")
plt.tight_layout()
plt.show()
```



```
[18]: import nbformat

input_path = "/content/drive/MyDrive/Colab Notebooks/Green Network.ipynb"
output_path = "/content/drive/MyDrive/Colab Notebooks/green26_0-4.ipnyb"

with open(input_path) as f:
    nb = nbformat.read(f, as_version=4)

# Filter: keep only cells that contain plotting code
figure_cells = [
    cell for cell in nb.cells
    if cell.cell_type == "code" and ("#EXPORT" in cell.source)
]

nb.cells = figure_cells

with open(output_path, 'w') as f:
```

```
nbformat.write(nb, f)
```